SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT), PUNE

Department of Computer Science & Engineering · Academic Evaluation Guide

PROE: Peer Review Orchestration Engine

Subject: Database Management Systems Lab (CA-II Mini Project) | **Rubric:** 100% Aligned with Final Submitted Report (Page 34)

📋 **Presentation Strategy & Exact Report Alignment:** The speaking portions below map **strictly 1-to-1** with the *Key Technical Deliverables Table* on Page 34 of our official DBMS Report (`Peer Review Orchestration Engine.pdf`). Each team member has an equal 25% share (~3.5 to 4 minutes). This guide provides the zero-knowledge explanation, exact code implementations, word-for-word speaking scripts, and anticipated viva Q&A.

Student Name & PRN	Assigned Technical Role	Official Report Key Deliverables (Page 34)	Code / Files

1. KARAN DAVE
PRN: 26070122507
	Conceptual Modeling & Normalization Lead	Problem statement & 10 functional requirements; Chen-notation EER diagram with overlapping role specialization [O]; mapped formal Functional Dependencies (FD_1–FD_5); wrote mathematical proofs for 1NF through BCNF normalization.	`docs/requirements.md`
			`database/er-diagram.png`
			`database/dbms-report.md`

2. SUSHANT YADAV PRN: 25070122199	**Relational Schema & Storage Architecture**	Physical DDL tables in MySQL InnoDB; primary & candidate unique keys; foreign key cascade delete actions; domain rating CHECK constraints (1–5); designed B-Tree indexing on frequently queried columns.	`database/schema.sql` `database/dbms-report.md` (Sec 5)

3. SUMIT KUMAR MAHATO
PRN: 25070122195
	Complex Queries & Query Execution	Realistic benchmark seed dataset across all four tables; authored multi-table INNER and LEFT JOIN queries; grouping aggregations with conditional sums for accept/reject tallies; evaluated MySQL query execution plans (EXPLAIN).	`database/seed.sql`
			`database/queries.sql`
			`database/dbms-report.md` (Sec 6–7)

4. SHREYA S. GAWANDE
PRN: 25070122183
	Functions, Procedures & Triggers	Deterministic User-Defined Functions (UDFs) for score calculations; stored procedures with ACID transaction rollback handlers; conflict-of-interest trigger and automated audit triggers; MongoDB activity aggregation pipeline.	`database/functions.sql`
			`database/procedures.sql`
			`database/triggers.sql`
			`database/mongodb-demo.js`

SPEAKER 1: KARAN DAVE (PRN: 26070122507)

CONCEPTUAL MODELING & NORMALIZATION

Speaking Time: Minutes 0:00 – 3:45 | **Target Slides:** Slide 1 (Problem & Scope), Slide 2 (Chen EER Diagram), Slide 3 (FDs & Normalization Proofs 1NF–BCNF)

1. What You Need to Know (Zero-Knowledge Explanation)

CONCEPT 1: PROBLEM STATEMENT & EER ROLE SPECIALIZATION

PROE is a scientific publishing platform where researchers submit manuscripts and independent peers review them. In standard ER, every entity is separate. In **Enhanced ER (EER)**, we use **Superclass/Subclass Specialization**: `USER` is the superclass, which specializes into `Author` and `Reviewer`. Because an academic can author a manuscript and also act as a reviewer for a peer's manuscript, this is modeled as an **Overlapping constraint [O]** with **Partial Participation**.

CONCEPT 2: FUNCTIONAL DEPENDENCIES & MATHEMATICAL NORMALIZATION PROOFS

A Functional Dependency ($X \to Y$) means attribute X uniquely determines attribute Y.

- **1NF:** All attribute values are atomic; primary key defined. Eliminates repeating groups.
- **2NF:** 1NF + No Partial Dependencies (non-prime attributes must depend on the *entire* candidate key).
- **3NF:** 2NF + No Transitive Dependencies ($X \to Y \to Z$ where non-key depends on non-key).
- **BCNF:** For every dependency $X \to Y$, the determinant X must be a super key.

WHAT WE BUILT IN PROE (CONCRETE IMPLEMENTATION)

- **10 Functional Requirements:** Documented in `docs/requirements.md`.
- **Chen-Notation EER Diagram:** `database/er-diagram.png` showing `USER` specialized into `AUTHOR` (authors $1:N$ `PAPER`) and `REVIEWER` (evaluates $1:N$ `REVIEW`).
- **Formal FDs in Report:** $FD_1: \text{user_id} \to \{\text{name}, \text{email}, \text{password_hash}, \text{created_at}\}$
  $FD_2: \text{paper_id} \to \{\text{author_id}, \text{title}, \text{abstract}, \text{keywords}, \text{paper_url}, \text{created_at}\}$
  $FD_3: \text{review_id} \to \{\text{paper_id}, \text{reviewer_id}, \text{rating}, \text{comments}, \text{recommendation}, \text{created_at}\}$
  $FD_4: (\text{paper_id}, \text{reviewer_id}) \to \{\text{review_id}, \text{rating}, \text{comments}, \text{recommendation}, \text{created_at}\}$
  $FD_5: \text{email} \to \{\text{user_id}, \text{name}, \text{password_hash}, \text{created_at}\}$
- **Mathematical Proofs:** Proved in `database/dbms-report.md` Section 4 that every determinant is a super key, achieving full BCNF compliance.

2. Word-For-Word Presentation Speaking Script

SPEAKING SCRIPT — KARAN DAVE

"Good morning respected evaluators and professors. My name is **Karan Dave**, and I lead the **Conceptual Modeling and Normalization** architecture for project **PROE — Peer Review Orchestration Engine**.

[Slide 1: Problem Statement & Requirements]
In academic publishing, managing research papers requires absolute data consistency: authors submit manuscripts,

independent reviewers score them, and editors make acceptance decisions. As detailed in our requirements document (`docs/requirements.md`), we defined 10 core functional requirements ensuring strict referential integrity, anti-self-review safeguards, and verifiable audit logging.

[Slide 2: Chen-Notation Enhanced ER (EER) Modeling]
Moving to our conceptual design, we engineered the Chen-notation EER diagram shown on screen. A key highlight of our DBMS rubric is **Superclass/Subclass Specialization**. In scientific platforms, a user is not rigidly just a reader. We modeled `USER` as a generalized superclass with attributes like `user_id`, `name`, and candidate key `email`.

This superclass specializes into two subclasses: `Author` and `Reviewer`. Crucially, we enforced an **Overlapping constraint [O]**: a scholar can submit paper A as an `Author` while simultaneously reviewing paper B as a `Reviewer`. We enforce **Partial Participation** because a newly registered user may initially only browse papers, and we enforce the business rule that an author cannot review their own paper.

[Slide 3: Functional Dependencies & BCNF Normalization Proofs]
To eliminate data redundancy, we derived five formal Functional Dependencies, FD_1 through FD_5.
- We decomposed our entities into **1NF** by enforcing scalar, atomic attributes and primary keys.
- We satisfied **2NF** by removing partial dependencies: review ratings depend on the composite candidate key `(paper_id, reviewer_id)`, while paper titles depend strictly on `paper_id`.
- We advanced to **3NF and BCNF** by eliminating transitive dependencies: non-key attributes depend only on the primary key, and every functional determinant in our schema is a valid super key, mathematically proving zero anomaly vulnerability.

I will now hand over to **Sushant Yadav**, who will present our physical relational schema DDL and storage architecture."

Transition Cue: *"Now Sushant will demonstrate how we implemented this normalized architecture into physical MySQL InnoDB DDL tables, cascades, and B-Tree indexing."*

3. Top Examiner Viva Questions & Bulletproof Answers

VIVA Q&A FOR KARAN DAVE

Q1: Why did you choose an Overlapping [O] constraint instead of a Disjoint [D] constraint for User subclasses?
Answer: "Sir, in academic conferences, a professor authors their own research papers while also acting as a peer reviewer for other submissions. If we used Disjoint [D], a person could only ever be an author OR a reviewer, but never both. Overlapping [O] accurately models real-world academic workflows."

Q2: What is the formal difference between 3NF and BCNF? How does PROE satisfy BCNF?
Answer: "In 3NF, for any functional dependency $X \to Y$, either X is a super key OR Y is a prime attribute. BCNF is stricter: X MUST always be a super key, with no exceptions for prime attributes. In PROE, every determinant across our decomposed tables ($user_id$ in users, $paper_id$ in papers, $review_id$ and $(paper_id, reviewer_id)$ in reviews) is a super key, satisfying BCNF."

Q3: How are the Author and Reviewer subclasses mapped to relational tables in MySQL?
Answer: "Rather than creating redundant tables with duplicate login credentials, we mapped the superclass into a unified `users` table. Role specialization is established dynamically through foreign keys in `papers.author_id` and `reviews.reviewer_id` referencing `users.user_id`."

SPEAKER 2: SUSHANT YADAV (PRN: 25070122199)

RELATIONAL SCHEMA & STORAGE ARCHITECTURE

Speaking Time: Minutes 3:45 – 7:30 | **Target Slides:** Slide 4 (Physical DDL & InnoDB Engine), Slide 5 (Keys, Cascades & CHECK Constraints), Slide 6 (B-Tree Indexing Architecture)

1. What You Need to Know (Zero-Knowledge Explanation)

CONCEPT 1: WHAT IS DDL AND WHY SPECIFY MYSQL INNODB?

Data Definition Language (DDL) defines the database structure (tables, columns, constraints). We explicitly specify `ENGINE=InnoDB` because the default legacy MyISAM engine does NOT support foreign keys or ACID transactions! InnoDB provides row-level locking, crash recovery, and enforces referential integrity.

CONCEPT 2: KEYS, CASCADES AND CHECK CONSTRAINTS

- **Primary Key (PK):** Unique identifier for every row (e.g. `user_id INT UNSIGNED AUTO_INCREMENT`).
- **Candidate Key / Unique Constraint:** Alternative unique identifier (e.g. `email` and composite `(paper_id, reviewer_id)`).
- **ON DELETE CASCADE:** If an author's manuscript is deleted, MySQL automatically deletes its associated reviews cleanly.
- **CHECK Constraint:** Domain integrity rule: `CONSTRAINT chk_reviews_rating CHECK (rating >= 1 AND rating <= 5)` prevents illegal ratings.

WHAT WE BUILT IN PROE (CONCRETE IMPLEMENTATION)

- **Schema File:** `database/schema.sql`.
- **Database & Charset:** `CREATE DATABASE proe_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;` (supports all international scientific characters).
- **Referential Rules:** `CONSTRAINT fk_papers_author FOREIGN KEY (author_id) REFERENCES users(user_id) ON DELETE CASCADE ON UPDATE CASCADE;`
  `CONSTRAINT fk_reviews_reviewer FOREIGN KEY (reviewer_id) REFERENCES users(user_id) ON DELETE RESTRICT ON UPDATE CASCADE;` (prevents deleting active reviewers).
- **B-Tree Indexing:** Composite index `idx_reviews_paper_rating ON reviews(paper_id, rating)` and foreign key index `idx_papers_author ON papers(author_id)`.

2. Word-For-Word Presentation Speaking Script

SPEAKING SCRIPT — SUSHANT YADAV

"Thank you, Karan. Respected evaluators, my name is **Sushant Yadav**. I am responsible for our **Relational Schema & Storage Architecture**.

[Slide 4: Physical DDL Implementation & InnoDB Engine]
In `database/schema.sql`, we translated our normalized model into production MySQL 8.0 DDL scripts. We explicitly selected the **InnoDB storage engine** with `utf8mb4` character encoding. Unlike legacy MyISAM, InnoDB guarantees ACID transaction compliance, row-level locking concurrency, and strict foreign key referential integrity.

[Slide 5: Key Constraints, Cascades & Domain Integrity]

Our physical tables enforce multi-layered database integrity:
1. **Primary & Candidate Keys:** Every table features an unsigned integer surrogate primary key. In `users`, `email` is enforced as an alternative candidate key using `CONSTRAINT uk_users_email UNIQUE (email)`.
2. **Referential Cascades:** On papers, `CONSTRAINT fk_papers_author` implements `ON DELETE CASCADE`, ensuring that if a manuscript is removed, orphaned review rows are automatically cleaned up. Conversely, on reviews, `fk_reviews_reviewer` implements `ON DELETE RESTRICT` to prevent accidental deletion of registered peer reviewers with historical evaluations.
3. **Domain CHECK Constraints:** In the `reviews` table, we enforce `CONSTRAINT chk_reviews_rating CHECK (rating >= 1 AND rating <= 5)`, ensuring invalid scores are rejected at the database level.
4. **Business Candidate Key:** We enforced `CONSTRAINT uk_paper_reviewer UNIQUE (paper_id, reviewer_id)`, preventing a reviewer from scoring the same paper twice.

[Slide 6: B-Tree Indexing Architecture]
To ensure high-throughput query latency as the database grows to thousands of papers, we engineered targeted **B-Tree Indexes**. We placed an index on `papers(author_id)` for fast portfolio lookups, and created the composite covering index `idx_reviews_paper_rating` on `reviews(paper_id, rating)`, allowing MySQL to compute aggregate paper scores directly from the index tree without fetching table data pages from disk.

I will now hand over to **Sumit Kumar Mahato**, who will present our benchmark seed dataset, multi-table joins, and query execution plans."

Transition Cue: *"Now Sumit will demonstrate our benchmark seed dataset, multi-table joins, grouping aggregations, and MySQL execution plans."*

3. Top Examiner Viva Questions & Bulletproof Answers

VIVA Q&A FOR SUSHANT YADAV

Q1: Why did you specify `ENGINE=InnoDB` explicitly in your DDL script?

Answer: "MySQL supports multiple storage engines. The default MyISAM engine does NOT support foreign key constraints or ACID transactions. InnoDB supports ACID transactions, row-level locking, and enforces referential integrity through foreign keys like `ON DELETE CASCADE`."

Q2: What is the difference between ON DELETE CASCADE and ON DELETE RESTRICT in your schema?

Answer: "In our schema, if a paper is deleted, `ON DELETE CASCADE` deletes its child reviews because reviews cannot exist without a paper. But if someone tries to delete a user who has active reviews, `ON DELETE RESTRICT` blocks the deletion to preserve academic review history and prevent orphaned audit records."

Q3: What does the unique constraint `uk_paper_reviewer UNIQUE (paper_id, reviewer_id)` accomplish?

Answer: "It creates a composite unique candidate key ensuring that a reviewer can review a given manuscript at most once, preventing ballot stuffing or duplicate reviews."

SPEAKER 3: SUMIT KUMAR MAHATO (PRN: 25070122195)

COMPLEX QUERIES & QUERY EXECUTION

Speaking Time: Minutes 7:30 – 11:15 | **Target Slides:** Slide 7 (Seed Data in Dependency Order), Slide 8 (Multi-table Joins & Grouping), Slide 9 (Query Execution Plans / EXPLAIN)

1. What You Need to Know (Zero-Knowledge Explanation)

CONCEPT 1: SEED INSERTION DEPENDENCY ORDER

Foreign keys enforce parent-child relationships. You cannot insert a paper before its author exists, and you cannot insert a review before the paper and reviewer exist. Therefore, `database/seed.sql` executes in strict chronological dependency order: `users` $\to$ `papers` $\to$ `reviews` $\to$ `audit_logs`.

CONCEPT 2: INNER JOIN VS LEFT JOIN WITH GROUP AGGREGATIONS

- **INNER JOIN:** Only returns rows when there is a match in both tables. If a paper has no reviews, an INNER JOIN with reviews omits the paper entirely.
- **LEFT JOIN:** Returns ALL papers from the left table, even if they have 0 reviews. We combine this with `COALESCE(AVG(r.rating), 0.0)` and `COUNT(r.review_id)` to calculate live review metrics without dropping newly submitted papers!

WHAT WE BUILT IN PROE (CONCRETE IMPLEMENTATION)

- **Seed Catalog (`database/seed.sql`):** 6 verified academic researchers, 5 realistic research manuscripts (e.g. Distributed Consensus, Quantum Cryptography), and 7 peer reviews with editorial recommendations (`ACCEPT` , `REVISE` , `REJECT`).
- **Analytical Joins (`database/queries.sql`):**

```
SELECT p.paper_id, p.title, u.name AS author_name,
       COALESCE(ROUND(AVG(r.rating), 2), 0.0) AS avg_rating,
       COUNT(r.review_id) AS total_reviews,
       SUM(CASE WHEN r.recommendation = 'ACCEPT' THEN 1 ELSE 0 END) AS accept_votes,
       SUM(CASE WHEN r.recommendation = 'REJECT' THEN 1 ELSE 0 END) AS reject_votes
FROM papers p
INNER JOIN users u ON p.author_id = u.user_id
LEFT JOIN reviews r ON p.paper_id = r.paper_id
GROUP BY p.paper_id, p.title, u.name
ORDER BY p.created_at DESC;
```

- **Query Execution Plan (`EXPLAIN`):** Verified that index `idx_reviews_paper_rating` changes the execution plan from a full table scan (`ALL`) to an index lookup (`ref`).

2. Word-For-Word Presentation Speaking Script

SPEAKING SCRIPT — SUMIT KUMAR MAHATO

"Thank you, Sushant. Good morning professors, I am **Sumit Kumar Mahato**. I lead our **Complex Queries & Query Execution** engineering.

[Slide 7: Benchmark Seed Data in Dependency Order]

In `database/seed.sql`, we populated the system with a realistic academic research catalog. Crucially, relational integrity

dictates that data must be seeded in **dependency order**. We first insert independent parent records into `users` with bcrypt-hashed passwords. Next, we seed manuscripts into `papers` referencing valid author IDs. Finally, we insert peer reviews referencing existing paper IDs and reviewer IDs, ensuring no foreign key violations occur.

[Slide 8: Multi-Table Joins & Grouping Aggregations]
In `database/queries.sql`, we developed production SQL queries powering the PROE web portal:
1. **The Paper Feed Query:** Combines an `INNER JOIN` with `users` to retrieve the author's name and a `LEFT JOIN` with `reviews`. A LEFT JOIN is critical here because newly submitted manuscripts have zero reviews; an INNER JOIN would have incorrectly excluded them from the feed! We compute `AVG(rating)` wrapped in a `COALESCE` function and `COUNT(review_id)`, grouping by the paper and author.
2. **Editorial Decision Tally:** Using conditional sums: `SUM(CASE WHEN r.recommendation = 'ACCEPT' THEN 1 ELSE 0 END)`, allowing journal editors to instantly tally acceptance versus rejection votes across papers.

[Slide 9: Evaluating Execution Plans with EXPLAIN]
To verify query performance, we analyzed the MySQL query optimizer using `EXPLAIN` statements. Without indexing, computing average ratings requires a full table scan (`type: ALL`) over the reviews table. By applying our composite B-Tree index `idx_reviews_paper_rating`, the execution plan shifts to `type: ref`, filtering directly by `paper_id` and calculating averages entirely from the index tree, achieving $O(\log N)$ lookup performance.

I will now invite **Shreya Gawande** to present our programmable SQL functions, stored procedures, audit triggers, and MongoDB aggregation pipeline."

Transition Cue: *"Shreya will now present our programmable SQL routines, stored procedures with ACID rollback handlers, audit triggers, and NoSQL demonstration."*

3. Top Examiner Viva Questions & Bulletproof Answers

VIVA Q&A FOR SUMIT KUMAR MAHATO

Q1: Why did you use a LEFT JOIN instead of an INNER JOIN when querying reviews for papers?

Answer: "Sir, an INNER JOIN only returns rows where matching records exist in both tables. If a new paper has just been submitted and has zero reviews, an INNER JOIN would omit that paper completely from the public listing. A LEFT JOIN preserves all papers from the primary table, returning NULL for review columns which we handle using `COALESCE` to display an average rating of 0.0."

Q2: How do you verify that an index is actually being utilized by the MySQL query optimizer?

Answer: "We prepend the query with the `EXPLAIN` keyword. In the output table, the `type` column changes from `ALL` (full table scan) to `ref`, the `key` column displays our index name (`idx_reviews_paper_rating`), and the `rows` column drops significantly, proving only relevant index pages are examined."

Q3: What is the purpose of the `COALESCE` function?

Answer: "When calculating `AVG(rating)` for a paper with zero reviews, SQL produces `NULL`. `COALESCE(AVG(rating), 0.0)` substitutes the NULL with `0.0`, preventing application-level NullPointerExceptions in the Java backend."

SPEAKER 4: SHREYA S. GAWANDE (PRN: 25070122183)

FUNCTIONS, PROCEDURES & TRIGGERS

Speaking Time: Minutes 11:15 – 15:00 | **Target Slides:** Slide 10 (Scalar UDF & Score Calculation), Slide 11 (Stored Procedure & ACID Rollback), Slide 12 (Audit Triggers & MongoDB Aggregation)

1. What You Need to Know (Zero-Knowledge Explanation)

CONCEPT 1: STORED FUNCTION VS STORED PROCEDURE

- **User-Defined Function (UDF):** Takes input parameters and returns exactly *one scalar value*. It can be used directly inside a `SELECT` statement (e.g. `SELECT title, fn_get_paper_average_rating(paper_id) FROM papers`).
- **Stored Procedure:** A precompiled batch of SQL statements that can perform transactions, execute multiple queries, take `IN` / `OUT` parameters, and execute `COMMIT` or `ROLLBACK` if an error occurs.

CONCEPT 2: DATABASE TRIGGERS & AUTOMATED AUDIT LOGGING

A **Trigger** is an event-driven routine executed automatically by the DBMS engine whenever an `INSERT`, `UPDATE`, or `DELETE` occurs. In PROE, an `AFTER INSERT ON reviews` trigger automatically inserts an immutable record into `audit_logs` capturing the timestamp, table name, review ID, and rating awarded.

WHAT WE BUILT IN PROE (CONCRETE IMPLEMENTATION)

- **Scalar Function (`database/functions.sql`):** `fn_get_paper_average_rating(p_paper_id)` returning `DECIMAL(3,2)` with `DETERMINISTIC READS SQL DATA`.
- **Stored Procedure (`database/procedures.sql`):** `sp_submit_review` with transaction management:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
START TRANSACTION;
-- Anti-self-review check: IF v_author_id = p_reviewer_id THEN SIGNAL SQLSTATE '45000'...
-- Insert into reviews
COMMIT;
```

- **Conflict & Audit Triggers (`database/triggers.sql`):** `trg_audit_reviews_insert` executing `AFTER INSERT ON reviews`, automatically inserting into `audit_logs`.
- **MongoDB Script (`database/mongodb-demo.js`):** Connected to MongoDB Atlas, runs an Aggregation Pipeline grouping events by action type with average ratings.

2. Word-For-Word Presentation Speaking Script

SPEAKING SCRIPT — SHREYA GAWANDE

"Thank you, Sumit. Respected evaluators, my name is **Shreya Gawande**. I will conclude our presentation by presenting our **Functions, Stored Procedures, Triggers**, and our MongoDB NoSQL demonstration.

[Slide 10: User-Defined Functions (UDFs)]

A crucial requirement of our DBMS rubric is server-side programmability. In `database/functions.sql`, we created the deterministic scalar function `fn_get_paper_average_rating`. It takes a `paper_id`, queries the reviews table, applies `COALESCE(ROUND(AVG(rating), 2), 0.0)`, and returns a precision decimal. This allows frontend and Java servlets to compute ratings in-line within any SELECT statement without writing repetitive JOIN logic.

[Slide 11: Stored Procedures & ACID Transaction Rollback]
In `database/procedures.sql`, we implemented `sp_submit_review`. Unlike raw queries from an application, our procedure encapsulates full **ACID transaction management**:
1. It initiates with `START TRANSACTION`.
2. It enforces anti-self-review safeguards: querying the paper's author and raising a `SIGNAL SQLSTATE '45000'` error if an author attempts to review their own submission.
3. Crucially, we declared an `EXIT HANDLER FOR SQLEXCEPTION` that automatically triggers a `ROLLBACK` if a network failure or foreign key constraint error occurs, guaranteeing zero partial writes.

[Slide 12: Automated Triggers & MongoDB Aggregation Pipeline]
In `database/triggers.sql`, we implemented the `trg_audit_reviews_insert` trigger operating on `AFTER INSERT ON reviews FOR EACH ROW`. It automatically captures the newly inserted review ID and records a tamper-proof timestamped entry into `audit_logs` without requiring application-level intervention.

Finally, in `database/mongodb-demo.js`, we satisfied the syllabus requirements for NoSQL databases. We demonstrated embedded document modeling for activity streams and executed an **Aggregation Pipeline**: filtering events with `$match`, calculating review distributions using `$group` with `$avg`, and sorting with `$sort`.

This completes our integrated database architecture. Our entire codebase, DDL scripts, ER diagrams, and normalization documentation are archived in our GitHub repository. We are now ready for your questions. Thank you!"

3. Top Examiner Viva Questions & Bulletproof Answers

VIVA Q&A FOR SHREYA GAWANDE

Q1: What is the difference between a Trigger and a Stored Procedure?
Answer: "A Stored Procedure is executed explicitly when called by an application using the `CALL` statement. A Trigger cannot be called manually; it is implicitly executed by the database engine in response to a specific DML event like `INSERT`, `UPDATE`, or `DELETE`."

Q2: Why do you need an EXIT HANDLER in a stored procedure?
Answer: "An `EXIT HANDLER FOR SQLEXCEPTION` acts as a safety catch block. If any SQL error occurs during the transaction (such as a foreign key or check constraint failure), the handler catches the exception, executes `ROLLBACK` to revert changes, and prevents partial corrupted data writes."

Q3: When would you use MongoDB instead of MySQL in a project like PROE?
Answer: "MySQL is essential for the core relational domain (papers, users, reviews) where strict referential integrity, ACID transactions, and normalization are required. MongoDB is optimal for unstructured, high-throughput, rapidly evolving data, such as real-time user activity logs, clickstream tracking, and system telemetry where schemas change frequently and horizontal sharding is beneficial."